Отчёт по лабораторной работе № 4

Продвинутое использование git

Савкин Дмитрий Вадимович

2026-10-12

Содержание

1 Введение **6**
1.1 Актуальность 6
1.2 Цель работы 6
1.3 Задачи 6
1.4 Объект и предмет исследования 6
1.5 Методы исследования 7
1.6 Схема выполнения работы 7

2 Теоретическая часть **8**

3 Материалы и методы **9**

4 Выполнение лабораторной работы **10**

5 Результаты **20**

6 Обсуждение результатов **21**

7 Заключение **22**

Список литературы **23**

Список иллюстраций

4.1 Включаем COPR-репозиторий и устанавливаем gitflow: dnf copr enable elegos/gitflow, dnf install gitflow 10
4.2 Устанавливаем Node.js и pnpm, настраиваем окружение и глобально ставим commitizen: dnf install nodejs, dnf install pnpm, pnpm setup, source ~/.bashrc, pnpm add -g commitizen 11
4.3 Устанавливаем standard-changelog и создаём тестовый репозиторий: pnpm add -g standard-changelog, mkdir ~/git-extended && cd ~/git-extended, git init -b master, echo "test" > readme.txt 11
4.4 Добавляем файл в индекс и устанавливаем адаптер commitizen для формата коммитов: git add readme.txt, pnpm add -g cz-conventional-changelog 12
4.5 Инициализируем package.json: pnpm init 12
4.6 Редактируем package.json, добавляя блок config.commitizen с путём к адаптеру cz-conventional-changelog, а также поля repository и author 13
4.7 Добавляем все файлы и создаём первый коммит через commitizen вместо обычного git commit: git add ., git cz — выбираем тип изменения feat 13
4.8 Проходим весь диалог commitizen: область изменения, краткое и подробное описание коммита, отметка о breaking changes 14
4.9 Отправляем коммит на GitHub и инициализируем git-flow: git push, git flow init (префикс тегов v) 14
4.10 Проверяем текущую ветку, отправляем весь репозиторий в хранилище и начинаем релиз версии 1.0.0: git branch, git push --all, git flow release start 1.0.0, генерируем журнал изменений standard-changelog --first-release, добавляем и коммитим CHANGELOG.md 15
4.11 Завершаем релиз: git flow release finish 1.0.0, вводим сообщение для тега Release 1.0.0 15
4.12 Отправляем ветки и теги на GitHub: git push --all, git push --tags 16
4.13 Создаём релиз на GitHub с описанием из журнала изменений: gh release create v1.0.0 -F CHANGELOG.md 16
4.14 Создаём ветку для новой функциональности: git flow feature start feature_branch 17

4.15 Завершаем работу над функциональностью и начинаем второй релиз версии 1.2.3: `git flow feature finish feature_branch`, `git flow release start 1.2.3` 17
4.16 Завершаем второй релиз, вводим сообщение для тега Release 1.2.3: `git flow release finish 1.2.3` 18
4.17 Отправляем ветки и теги второго релиза на GitHub: `git push --all`, `git push --tags` 18
4.18 Создаём второй релиз на GitHub: `gh release create v1.2.3 -F CHANGELOG.md` 19

Список таблиц

1 Введение

1.1 Актуальность

Модель git-flow и Conventional Commits — общепринятая практика организации командной разработки с предсказуемой историей релизов.

1.2 Цель работы

Получение навыков правильной работы с репозиториями git.

1.3 Задачи

- Выполнить работу для тестового репозитория.
- Преобразовать рабочий репозиторий в репозиторий с git-flow и conventional commits.

1.4 Объект и предмет исследования

Объект — git-репозиторий; предмет — модель ветвления git-flow и формат коммитов Conventional Commits.

1.5 Методы исследования

Эмпирический метод: практическое выполнение заданий в виртуальной машине Fedora Linux с фиксацией результатов скриншотами и видеозаписью.

1.6 Схема выполнения работы

Работа выполнялась по стандартной схеме курса: выполнение действия в виртуальной машине, фиксация скриншотом, занесение в отчёт, публикация в git-репозиторий отдельным релизом.

2 Теоретическая часть

Git-flow — модель ветвления для Git (Vincent Driessen), разделяющая разработку на долгоживущие ветки master/develop и вспомогательные ветки feature/release/hotfix. Conventional Commits — соглашение об оформлении сообщений коммитов (`<тип>(<область>): <описание>`), облегчающее автоматическую генерацию журнала изменений (changelog) и семантическое версионирование.

Более подробно см. в [1].

3 Материалы и методы

Используемое ПО и оборудование: хост-машина под управлением Windows, гипервизор VirtualBox, виртуальная машина с Fedora Linux (Sway), стандартные утилиты дистрибутива.

4 Выполнение лабораторной работы

Включаем COPR-репозиторий и устанавливаем `gitflow`: `dnf copr enable elegos/gitflow`, `dnf install gitflow`.

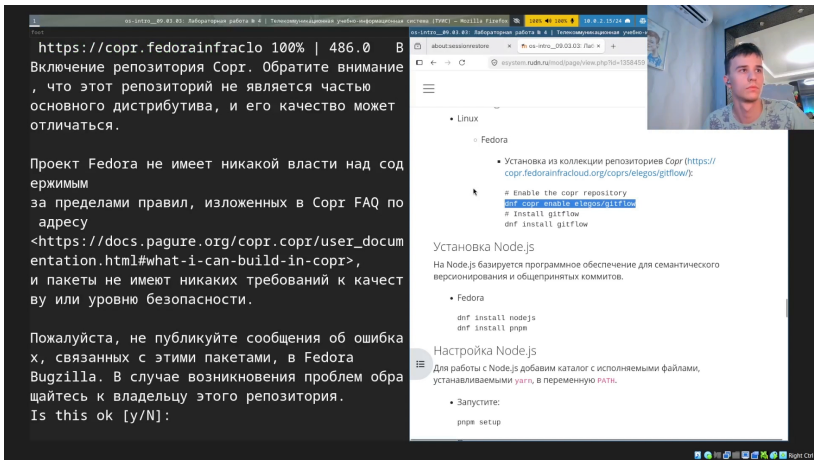


Рисунок 4.1: Включаем COPR-репозиторий и устанавливаем `gitflow`: `dnf copr enable elegos/gitflow`, `dnf install gitflow`

Устанавливаем Node.js и pnpm, настраиваем окружение и глобально ставим `commitizen`: `dnf install nodejs`, `dnf install pnpm`, `pnpm setup`, `source ~/.bashrc`, `pnpm add -g commitizen`.

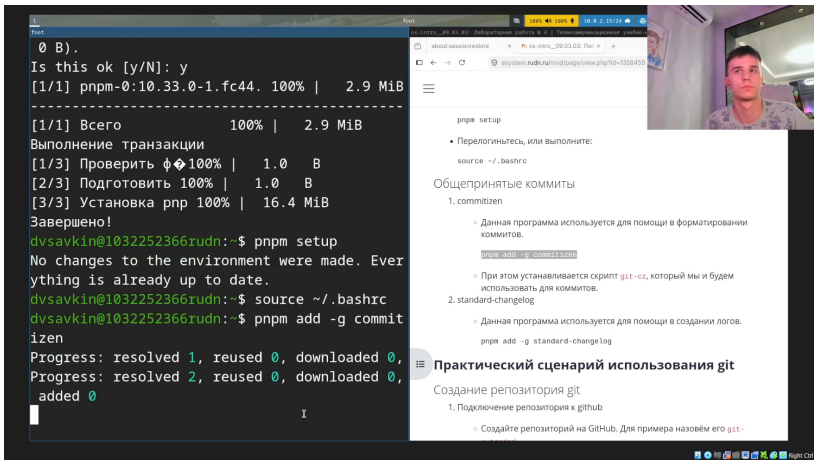


Рисунок 4.2: Устанавливаем Node.js и pnpm, настраиваем окружение и глобально ставим commitizen: `dnf install nodejs`, `dnf install pnpm`, `pnpm setup`, `source ~/.bashrc`, `pnpm add -g commitizen`

Устанавливаем standard-changelog и создаём тестовый репозиторий: `pnpm add -g standard-changelog`, `mkdir ~/git-extended && cd ~/git-extended`, `git init -b master`, `echo "test" > readme.txt`.

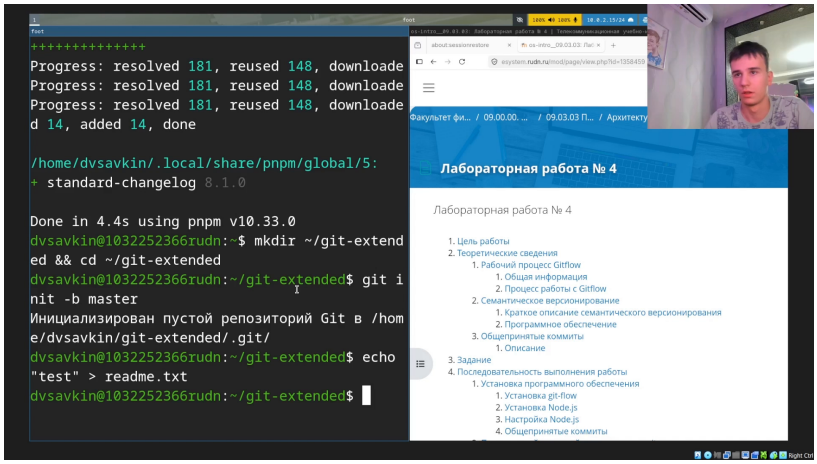


Рисунок 4.3: Устанавливаем standard-changelog и создаём тестовый репозиторий: `pnpm add -g standard-changelog`, `mkdir ~/git-extended && cd ~/git-extended`, `git init -b master`, `echo "test" > readme.txt`

Добавляем файл в индекс и устанавливаем адаптер commitizen для формата коммитов: `git add readme.txt`, `pnpm add -g cz-conventional-changelog`.

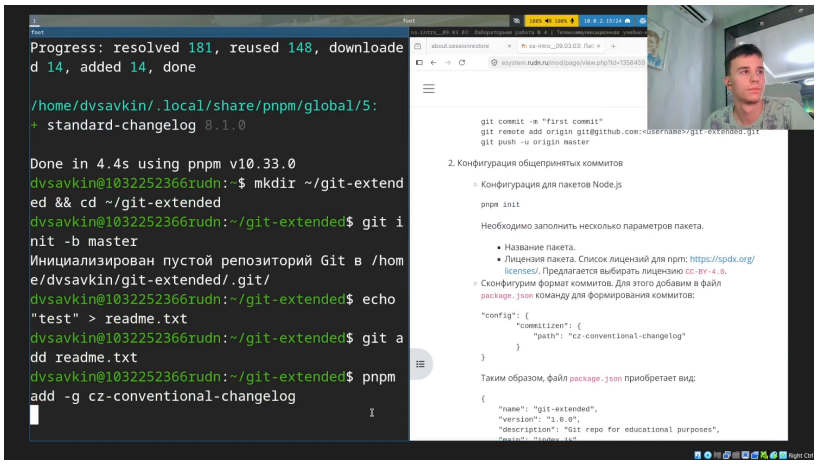


Рисунок 4.4: Добавляем файл в индекс и устанавливаем адаптер commitizen для формата коммитов: `git add readme.txt`, `pnpm add -g cz-conventional-changelog`

Инициализируем `package.json`: `pnpm init`.

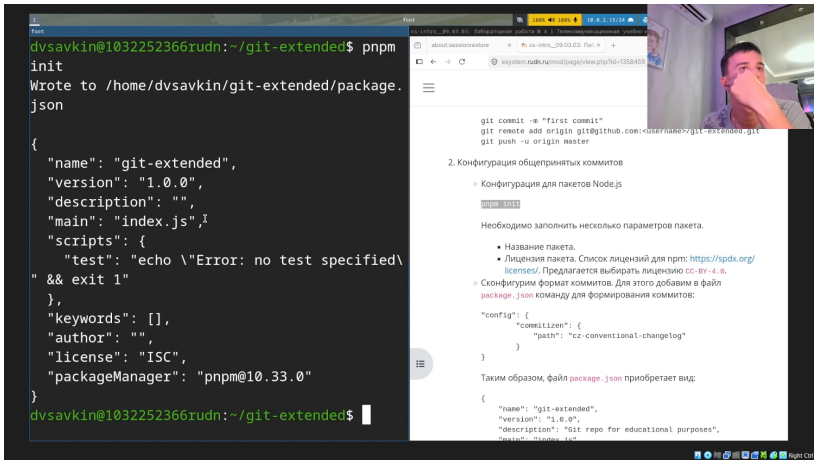


Рисунок 4.5: Инициализируем `package.json`: `pnpm init`

Редактируем `package.json`, добавляя блок `config.commitizen` с путём к адаптеру `cz-conventional-changelog`, а также поля `repository` и `author`.

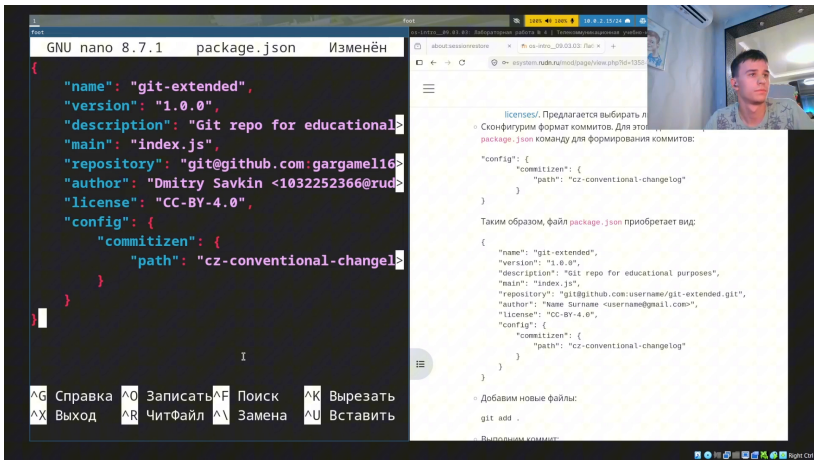


Рисунок 4.6: Редактируем `package.json`, добавляя блок `config.commitizen` с путём к адаптеру `cz-conventional-changelog`, а также поля `repository` и `author`

Добавляем все файлы и создаём первый коммит через commitizen вместо обычного `git commit`: `git add .`, `git cz` — выбираем тип изменения `feat`.

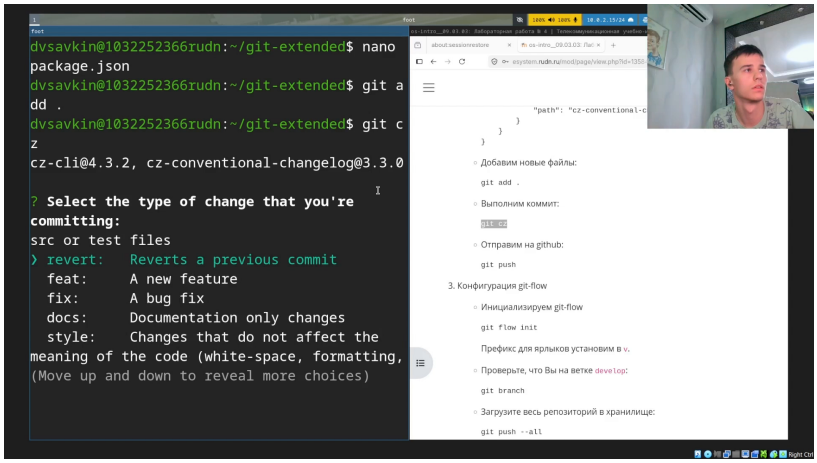


Рисунок 4.7: Добавляем все файлы и создаём первый коммит через commitizen вместо обычного `git commit`: `git add .`, `git cz` — выбираем тип изменения `feat`

Проходим весь диалог commitizen: область изменения, краткое и подробное описание коммита, отметка о breaking changes.

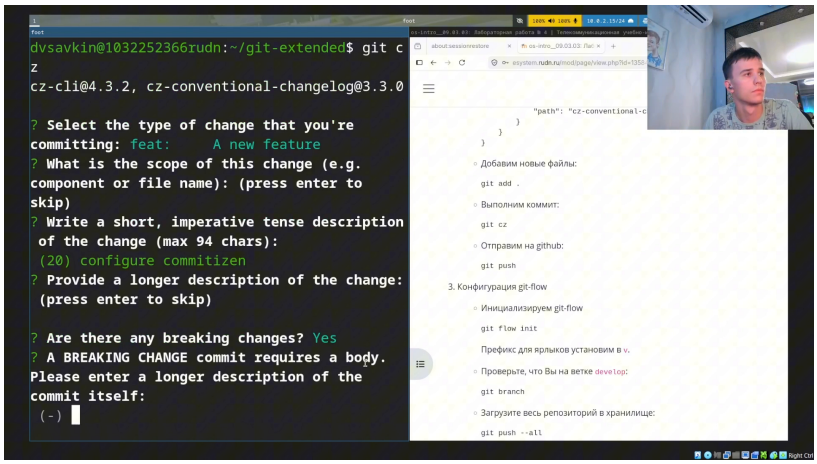


Рисунок 4.8: Проходим весь диалог commitizen: область изменения, краткое и подробное описание коммита, отметка о breaking changes

Отправляем коммит на GitHub и инициализируем git-flow: `git push`, `git flow init` (префикс тегов `v`).

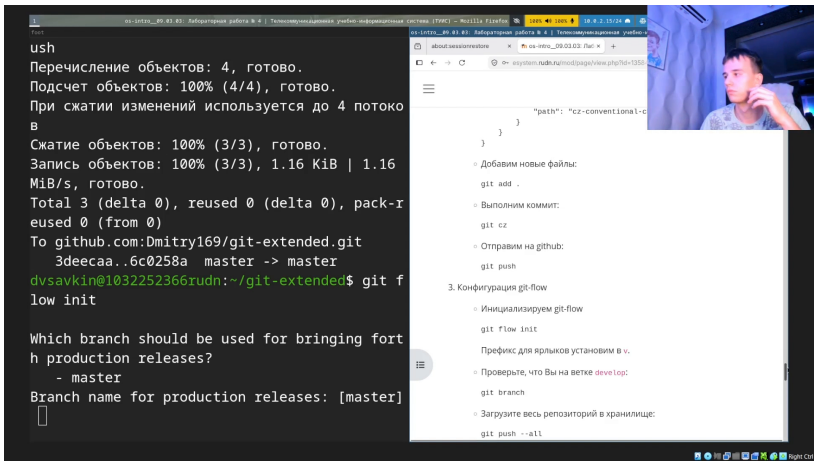


Рисунок 4.9: Отправляем коммит на GitHub и инициализируем git-flow: `git push`, `git flow init` (префикс тегов `v`)

Проверяем текущую ветку, отправляем весь репозиторий в хранилище и начинаем релиз версии 1.0.0: `git branch`, `git push --all`, `git flow release start 1.0.0`, генерируем журнал изменений `standard-changelog --first-release`, добавляем и коммитим `CHANGELOG.md`.

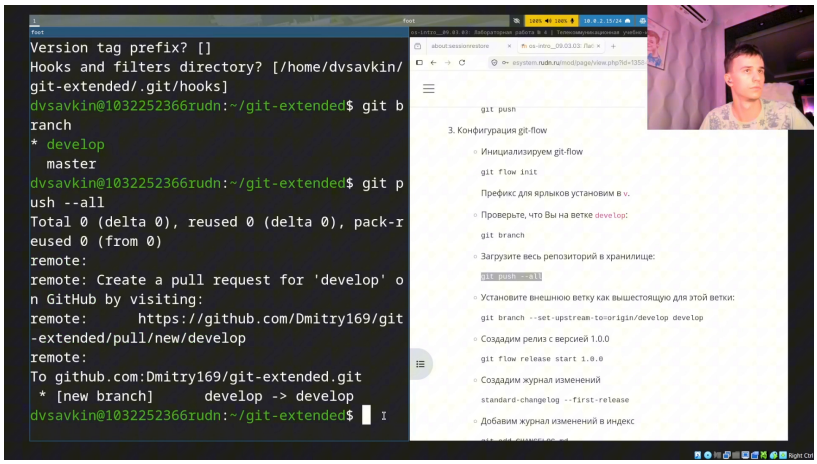


Рисунок 4.10: Проверяем текущую ветку, отправляем весь репозиторий в хранилище и начинаем релиз версии 1.0.0: `git branch`, `git push --all`, `git flow release start 1.0.0`, генерируем журнал изменений `standard-changelog --first-release`, добавляем и коммитим `CHANGELOG.md`

Завершаем релиз: `git flow release finish 1.0.0`, вводим сообщение для тега `Release 1.0.0`.

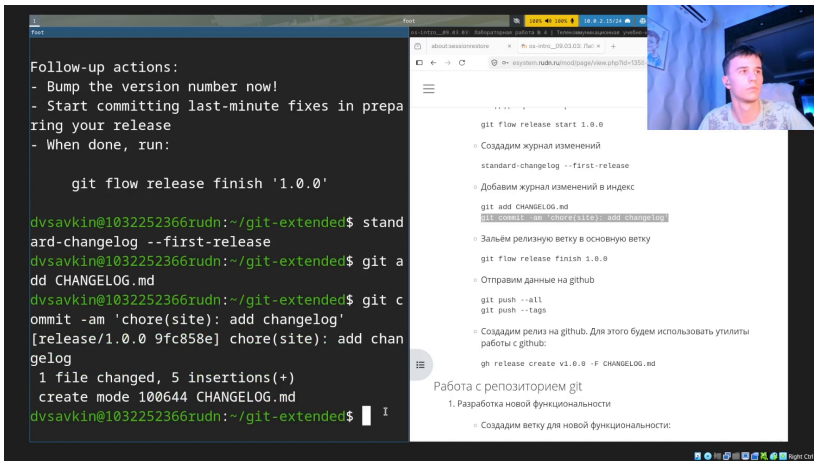


Рисунок 4.11: Завершаем релиз: `git flow release finish 1.0.0`, вводим сообщение для тега `Release 1.0.0`

Отправляем ветки и теги на GitHub: `git push --all`, `git push --tags`.

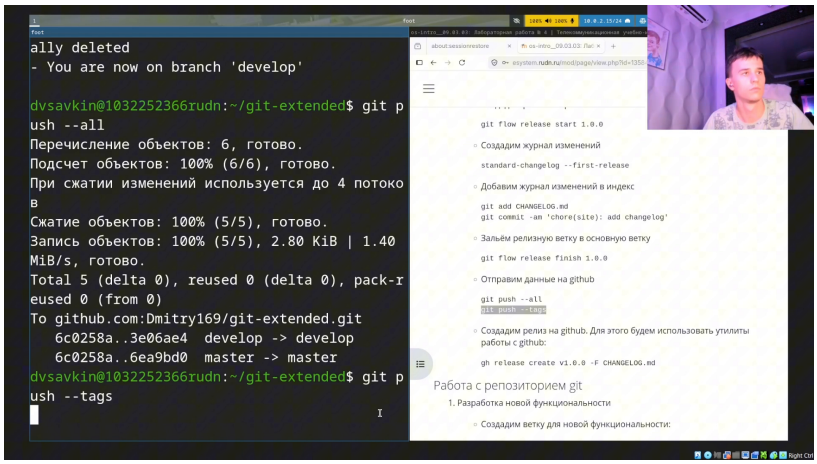


Рисунок 4.12: Отправляем ветки и теги на GitHub: `git push --all`, `git push --tags`

Создаём релиз на GitHub с описанием из журнала изменений: `gh release create v1.0.0 -F CHANGELOG.md`.

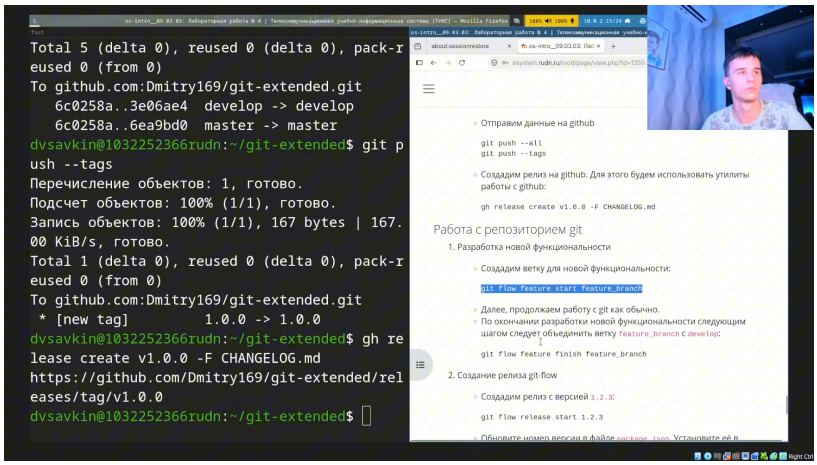


Рисунок 4.13: Создаём релиз на GitHub с описанием из журнала изменений: `gh release create v1.0.0 -F CHANGELOG.md`

Создаём ветку для новой функциональности: `git flow feature start feature_branch`.

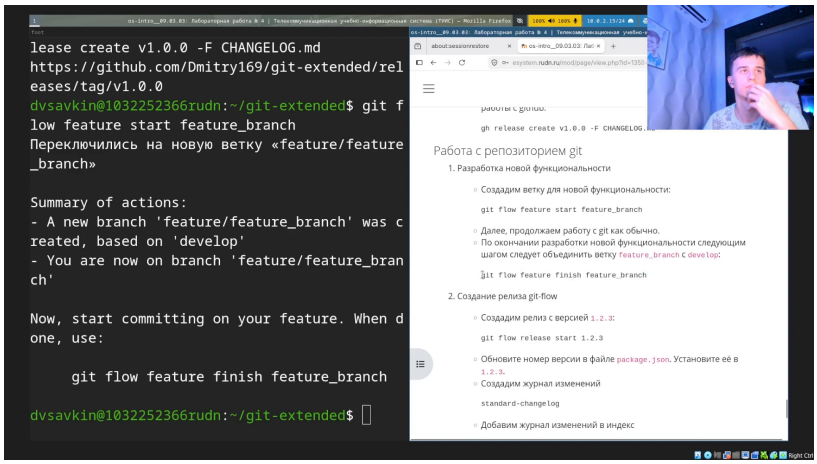


Рисунок 4.14: Создаём ветку для новой функциональности: `git flow feature start feature_branch`

Завершаем работу над функциональностью и начинаем второй релиз версии 1.2.3: `git flow feature finish feature_branch`, `git flow release start 1.2.3`.

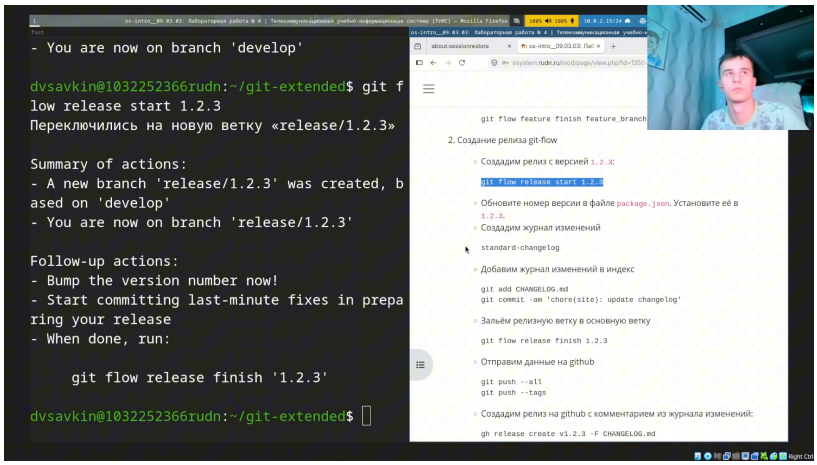


Рисунок 4.15: Завершаем работу над функциональностью и начинаем второй релиз версии 1.2.3: `git flow feature finish feature_branch`, `git flow release start 1.2.3`

Завершаем второй релиз, вводим сообщение для тега `Release 1.2.3`: `git flow release finish 1.2.3`.

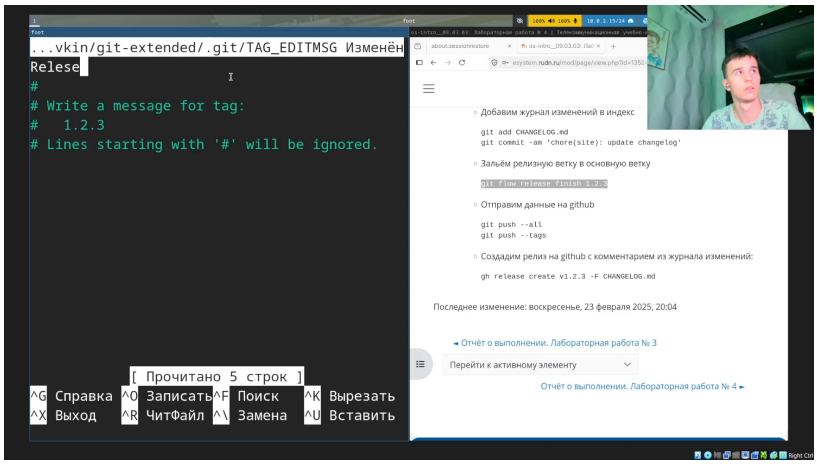


Рисунок 4.16: Завершаем второй релиз, вводим сообщение для тега Release 1.2.3: `git flow release finish 1.2.3`

Отправляем ветки и теги второго релиза на GitHub: `git push --all`, `git push --tags`.

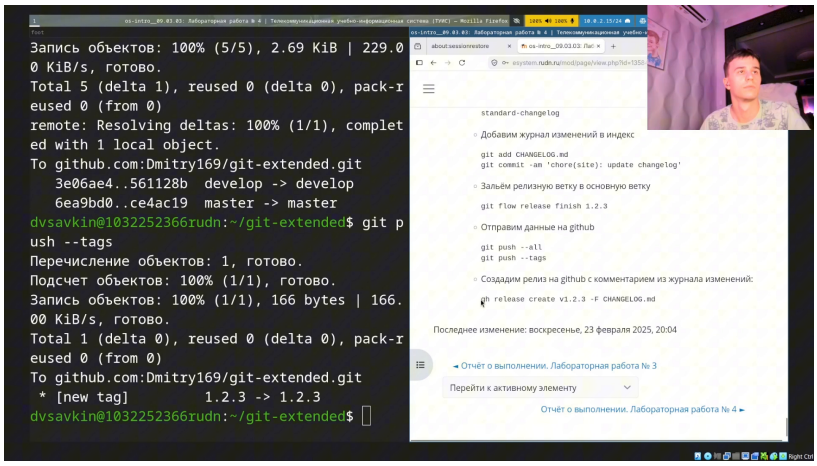


Рисунок 4.17: Отправляем ветки и теги второго релиза на GitHub: `git push --all`, `git push --tags`

Создаём второй релиз на GitHub: `gh release create v1.2.3 -F CHANGELOG.md`.

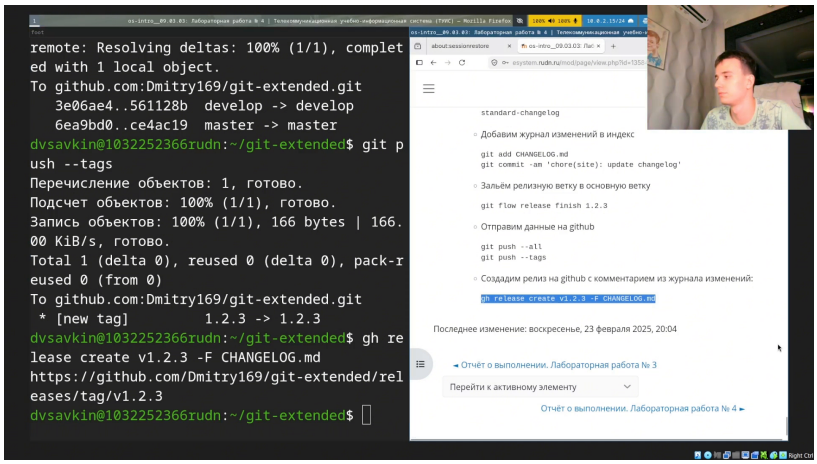


Рисунок 4.18: Создаём второй релиз на GitHub: `gh release create v1.2.3 -F CHANGELOG.md`

5 Результаты

В ходе выполнения работы получены следующие объективные результаты: Изучена и опробована на практике модель ветвления Gitflow и принцип общепринятых коммитов (Conventional Commits). Настроены git-flow и commitizen в тестовом репозитории, проведён полный цикл разработки новой функциональности через feature-ветку и два цикла выпуска релизов (1.0.0 и 1.2.3) с автоматической генерацией журнала изменений и публикацией релизов на GitHub.

6 Обсуждение результатов

Применение git-flow и Conventional Commits на тестовом репозитории подтвердило ожидаемое поведение: чёткое разделение веток и автоматическая генерация журнала изменений.

7 Заключение

Изучена и опробована на практике модель ветвления Gitflow и принцип общепринятых коммитов (Conventional Commits). Настроены git-flow и commitizen в тестовом репозитории, проведён полный цикл разработки новой функциональности через feature-ветку и два цикла выпуска релизов (1.0.0 и 1.2.3) с автоматической генерацией журнала изменений и публикацией релизов на GitHub.

Список литературы

1. *Software Freedom Conservancy*. Pro Git.. — 2014.

Ответы на контрольные вопросы

1. *Software Freedom Conservancy*. Pro Git.. — 2014.